

Documentación técnica exhaustiva

Explicación completa del código del proyecto OLLAMA_RAG

Este documento explica de forma detallada toda la lógica implementada en el proyecto: limpieza del dataset, clasificación temática heurística, generación de embeddings, construcción del índice vectorial, pipeline RAG con LangChain y Ollama, e interfaz local con Gradio.

Proyecto

Sistema RAG con Ollama, LangChain y Gradio

Código explicado

8 archivos fuente y de configuración

Entrega

Documento PDF con anexo de código completo

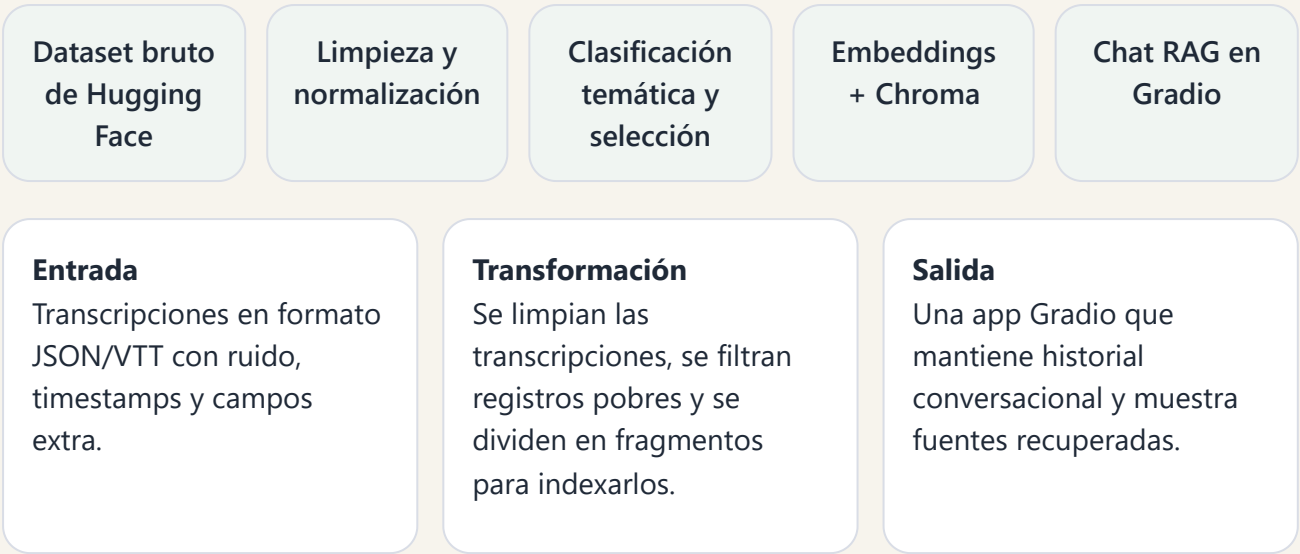
Índice

1. Visión general del sistema
2. Mapa completo de archivos
3. Flujo de ejecución de extremo a extremo
4. Archivos de configuración y dependencias
5. Explicación completa de `clean_json.py`
6. Explicación completa de `index_data.py`
7. Explicación completa de `rag_utils.py`
8. Explicación completa de `app.py`
9. Decisiones de diseño, límites y mejoras posibles
10. Anexo: código fuente completo

1. Visión general del sistema

El proyecto implementa un sistema RAG local orientado a transcripciones de Khan Academy. El objetivo es permitir que un usuario formule preguntas en lenguaje natural y que el sistema

responda únicamente con información recuperada del dataset limpio. Toda la cadena se ejecuta en local: limpieza, clasificación temática, embeddings, búsqueda semántica y generación de respuestas.



Idea clave: el proyecto separa claramente tres responsabilidades: preparación de datos, indexación vectorial y explotación del índice mediante una interfaz conversacional.

2. Mapa completo de archivos

Archivo	Líneas	Rol dentro del sistema
.gitignore	6	
.env.example	4	
requirements.txt	10	
clean_json.py	87	
index_data.py	100	
rag_utils.py	515	
app.py	389	
README.md	134	

- `.gitignore` evita subir entorno virtual, cachés, índice vectorial y variables sensibles.
- `.env.example` documenta las variables de entorno que controlan la conexión a Ollama y los modelos por defecto.
- `requirements.txt` fija las bibliotecas necesarias para ejecutar el proyecto.
- `clean_json.py` descarga o lee el dataset y produce `train_clean.json`.

- `index_data.py` crea el almacén vectorial persistente con Chroma y Ollama embeddings.
- `rag_utils.py` concentra la lógica compartida: limpieza, deduplicación, clasificación, chunking, selección de modelos y formateo de contexto.
- `app.py` define el pipeline RAG, el prompt, la memoria conversacional y la interfaz Gradio.
- `README.md` describe uso, instalación, ejecución y evidencias.

3. Flujo de ejecución de extremo a extremo

3.1 Flujo de preparación

1. Se ejecuta `clean_json.py`.
2. El script descarga el split indicado de Hugging Face o lee un JSON local.
3. Cada registro pasa por `normalize_record()`, que limpia texto, normaliza campos y asigna tema.
4. Se eliminan duplicados y, si procede, se selecciona un subconjunto balanceado por tema.
5. El resultado se serializa a `train_clean.json`.

3.2 Flujo de indexación

1. Se ejecuta `index_data.py`.
2. El script lee `train_clean.json` y convierte cada transcripción en documentos fragmentados.
3. Se crea un objeto `OllamaEmbeddings` y una colección persistente Chroma.
4. Los documentos se insertan por lotes y se guarda un manifiesto con metadatos del índice.

3.3 Flujo de consulta

1. El usuario abre `app.py` y escribe una pregunta.
2. La aplicación carga el índice Chroma y aplica búsqueda semántica con filtros opcionales.
3. Los fragmentos recuperados se convierten en bloque de contexto mediante `build_context_block()`.
4. LangChain compone el prompt final usando sistema + historial + pregunta actual.
5. Ollama genera la respuesta con el modelo elegido.
6. La interfaz actualiza el chat y muestra las fuentes usadas.

4. Archivos de configuración y dependencias

4.1 .gitignore

Este archivo protege el repositorio de artefactos locales y pesados. Excluye el entorno virtual, cachés de Python, la carpeta persistente del vector store y el archivo `.env`, que podría contener configuraciones sensibles del entorno.

- `.venv/`: evita subir paquetes y ejecutables locales.
- `__pycache__`/ y `.pytest_cache/`: evitan archivos temporales.
- `vectorstore/`: impide versionar la base vectorial generada en local.
- `.env`: evita exponer configuración personalizada del usuario.

4.2 `.env.example`

Actúa como plantilla. No se usa directamente, pero enseña qué variables acepta el proyecto y qué valores esperan las partes principales del sistema. Sirve para replicar el entorno de ejecución.

- `OLLAMA_BASE_URL`: dirección del servidor de Ollama.
- `OLLAMA_LLM_MODEL`: modelo generativo para responder preguntas.
- `OLLAMA_EMBED_MODEL`: modelo de embeddings usado al indexar o consultar.

4.3 `requirements.txt`

Fija dependencias concretas para minimizar diferencias de comportamiento entre entornos. Cada paquete cumple un papel claro.

`chromadb` `datasets` `gradio` `langchain` `langchain-chroma` `langchain-community`
`langchain-ollama` `langchain-text-splitters` `python-dotenv`

- `datasets` permite descargar y materializar el dataset de Hugging Face.
- `langchain` aporta la composición del prompt, mensajes y parser de salida.
- `langchain-ollama` conecta LangChain con modelos locales servidos por Ollama.
- `langchain-chroma` encapsula el acceso a la base vectorial Chroma.
- `langchain-text-splitters` divide transcripciones largas en chunks.
- `gradio` construye la interfaz del chatbot.
- `python-dotenv` carga variables desde archivo `.env`.

4.4 `README.md`

Aunque no es lógica ejecutable, forma parte importante de la solución. Documenta instalación, flujo de uso, variables de entorno y capturas de funcionamiento. En términos de ingeniería, este archivo es el punto de entrada humano al proyecto.

5. Explicación completa de `clean_json.py`

Este script es el primer escalón del pipeline. Su misión es transformar el dataset original en una versión utilizable para RAG: homogénea, limpia, con campos relevantes y con una etiqueta temática útil para filtrar después en la app.

5.1 Importaciones

- `argparse` se usa para convertir el script en una herramienta de línea de comandos configurable.
- `Counter` permite resumir cuántos registros quedaron por tema al final del proceso.
- `Path` estandariza rutas de archivo.
- `load_dataset` descarga el dataset remoto desde Hugging Face.
- Desde `rag_utils` se reutiliza toda la lógica de normalización y escritura.

5.2 `parse_args()`

Define la interfaz CLI del script. Es importante porque convierte el limpiador en una herramienta reproducible, no en un script rígido.

- `--dataset-name` y `--split` permiten elegir dataset y partición.
- `--input-json` permite saltarse Hugging Face y usar un archivo local.
- `--output` fija la ruta del JSON limpio.
- `--max-records` limita el tamaño del subconjunto final.
- `--min-words` define el umbral mínimo de utilidad para una transcripción.
- `--seed` hace reproducible la selección balanceada por tema.

5.3 `load_source_records(args)`

Centraliza la lectura de la fuente de datos. Si el usuario pasa un JSON local, el script acepta dos formatos: una lista directa o un objeto con clave `train`. Si no se proporciona `--input-json`, se usa `load_dataset()` y se materializa el split completo como una lista de diccionarios de Python.

Ventaja: el mismo pipeline sirve tanto para desarrollo con Hugging Face como para repetir experimentos offline con archivos previamente descargados.

5.4 `main()`

Es el orquestador real de la limpieza. El flujo interno es lineal y fácil de seguir:

1. Recoge argumentos.
2. Carga registros crudos.
3. Aplica `normalize_record()` uno a uno.
4. Descarta `None`, que representa registros inválidos.
5. Deduplica con `deduplicate_records()`.
6. Selecciona subconjunto con `choose_balanced_subset()`.
7. Escribe el JSON final con `write_json()`.
8. Calcula estadísticas por tema para que el usuario valide visualmente el resultado.

La filosofía del script es separar orquestación y lógica. Aquí casi no hay reglas de negocio complejas; la mayoría viven en `rag_utils.py`. Eso facilita mantenimiento y pruebas.

6. Explicación completa de `index_data.py`

Este archivo toma el dataset ya limpio y lo convierte en un índice vectorial persistente. Su responsabilidad no es decidir qué limpiar, sino preparar los datos para búsqueda semántica eficiente.

6.1 Importaciones

- `argparse` para parametrizar la indexación.
- `time` para sellar el manifiesto con la fecha de creación.
- `Chroma` y `OllamaEmbeddings` para crear y poblar la base vectorial.
- Desde `rag_utils` se reutilizan rutas por defecto, chunking, lectura del dataset y escritura del manifiesto.

6.2 `parse_args()`

Define los parámetros ajustables de la indexación.

- `--input`: JSON limpio de entrada.
- `--persist-dir`: carpeta donde se escribirá Chroma.
- `--collection-name`: nombre lógico de la colección.
- `--chunk-size` y `--chunk-overlap`: controlan granularidad y solape de los fragmentos.
- `--embedding-model`: permite cambiar el modelo sin editar código.
- `--ollama-base-url`: permite apuntar a otro host de Ollama si fuera necesario.

6.3 `main()`

Validación de entrada

Antes de nada, comprueba que el JSON limpio existe. Si no existe, corta la ejecución con un mensaje claro, obligando a ejecutar primero `clean_json.py`.

Carga y fragmentación

Lee los registros limpios y llama a `build_documents()`. Esta función crea objetos `Document` de `LangChain`, uno por fragmento. Aquí sucede la traducción entre “transcripción limpia” y “unidad indexable”.

Preparación segura del directorio

`ensure_safe_output_dir()` elimina el índice anterior y crea de nuevo la carpeta de salida, pero solo si la ruta está dentro del proyecto. Esta defensa evita borrar accidentalmente directorios ajenos

por un error de configuración.

Embeddings e inserción por lotes

Se crea un objeto `OllamaEmbeddings` y una instancia de `Chroma`. Después, los documentos se insertan por lotes de 64. Este tamaño es un compromiso razonable: reduce overhead respecto a insertar uno a uno, pero sin crear lotes excesivamente grandes que puedan ser más frágiles.

Manejo de errores

Si Ollama no está levantado o el modelo no existe, el script captura la excepción y termina con un mensaje prescriptivo que sugiere el comando `ollama pull nomic-embed-text:latest`.

Manifiesto

Al final se genera `index_manifest.json`, que guarda metadatos operativos: número de registros, número de chunks, tamaño de fragmento, modelo de embeddings y URL de Ollama. La app usa este manifiesto como referencia al arrancar.

7. Explicación completa de rag_utils.py

Este es el archivo más importante del proyecto desde el punto de vista de reutilización. Contiene constantes, expresiones regulares, heurísticas y funciones compartidas por el limpiador, el indexador y la aplicación web.

7.1 Constantes globales

Las constantes evitan números mágicos y centralizan configuración común.

- `DEFAULT_DATASET_NAME` y `DEFAULT_DATASET_SPLIT` fijan el origen por defecto del dataset.
- `DEFAULT_MIN_WORDS` define el umbral mínimo de utilidad de un registro.
- `DEFAULT_MAX_RECORDS` limita el subconjunto para desarrollo y demo.
- `DEFAULT_CHUNK_SIZE` y `DEFAULT_CHUNK_OVERLAP` regulan la fragmentación.
- `DEFAULT_COLLECTION_NAME`, `DEFAULT_CLEAN_PATH`, `DEFAULT_VECTORSTORE_DIR` y `DEFAULT_MANIFEST_NAME` normalizan nombres y rutas internas.
- `DEFAULT_OLLAMA_BASE_URL` fija la URL estándar del servidor local de Ollama.
- `TOPIC_ALL_LABEL` representa la opción de "sin filtro" en la interfaz.

7.2 Expresiones regulares

Estas regex representan el núcleo de la limpieza textual.

- `TIMESTAMP_PATTERN` detecta líneas de marcas de tiempo VTT.
- `CUE_INDEX_PATTERN` detecta índices numéricos de subtítulos.
- `TAG_PATTERN` elimina marcas como `[music]` o `[applause]`.
- `HTML_PATTERN` elimina etiquetas HTML incrustadas.

- `WHITESPACE_PATTERN` comprime espacios y saltos múltiples.
- `SENTENCE_SPLIT_PATTERN` separa frases para poder deduplicarlas de forma más semántica.

7.3 Diccionario `TOPIC_KEYWORDS`

Este diccionario implementa la clasificación temática heurística. Cada clave es una categoría de alto nivel y cada valor es una lista de palabras clave relacionadas. La función `infer_topic()` cuenta cuántas coincidencias tiene cada categoría en el título y un fragmento de la transcripción.

Esto no es un clasificador de machine learning. Es una regla local, transparente, barata y suficiente para habilitar un filtro temático en la interfaz sin necesidad de un segundo modelo.

7.4 Funciones de limpieza y normalización

`normalize_whitespace(text)`

Condensa secuencias de espacios en blanco en un único espacio y recorta extremos. Se usa en muchos puntos del proyecto para estabilizar cadenas antes de comparar o guardar.

`stable_text_id(*parts)`

Genera un identificador corto y estable a partir de varias cadenas concatenadas y hasheadas con SHA-1. No busca seguridad criptográfica, sino reproducibilidad y compacidad.

`dedupe_adjacent_sentences(text)`

Divide el texto por frases, normaliza cada frase y elimina repeticiones adyacentes. Esto es útil porque algunas transcripciones automáticas repiten líneas consecutivas.

`clean_transcript(raw_text)`

Es la función de limpieza más importante. Recorre línea por línea y descarta elementos sin valor semántico: cabeceras WEBVTT, metadatos, notas, timestamps y numeraciones. Después vuelve a limpiar el resultado total eliminando HTML, etiquetas entre corchetes, entidades HTML y guiones sueltos. Al final normaliza el espacio y elimina frases consecutivas repetidas.

Resultado esperado: una transcripción corrida, limpia y apta para ser troceada e indexada.

7.5 Clasificación y normalización de registros

`infer_topic(title, content)`

Construye una bolsa de texto con el título y un preview de la transcripción. Calcula puntuaciones por categoría contando coincidencias de palabras clave. Devuelve el tema con mayor puntuación, aplica una regla especial para títulos cortos de vocabulario y, si no hay señales suficientes, cae en `General`.

`normalize_record(raw_record, min_words)`

Convierte un registro crudo del dataset en un registro limpio y homogéneo.

- Normaliza el título.
- Limpia el contenido con `clean_transcript()`.
- Extrae `video_url` como URL principal y conserva la URL original de subtítulos en `subtitle_url`.
- Descarta registros sin título, sin contenido o con pocas palabras.
- Calcula un identificador estable.
- Devuelve un diccionario final con `id`, `title`, `content`, `url`, `subtitle_url`, `language`, `topic` y `word_count`.

`deduplicate_records(records)`

Elimina duplicados usando preferentemente la URL como huella. Si no hay URL, usa un hash derivado de título y contenido. Así evita indexar varias veces el mismo vídeo o una transcripción prácticamente igual.

`choose_balanced_subset(records, max_records, seed)`

Si el número de registros supera el máximo deseado, agrupa por tema y selecciona de forma rotatoria un subconjunto balanceado. El uso de una semilla fija garantiza reproducibilidad.

7.6 Construcción de documentos para LangChain

`build_documents(records, chunk_size, chunk_overlap)`

Transforma los registros limpios en objetos `Document`. Usa `RecursiveCharacterTextSplitter` con una jerarquía de separadores que intenta partir antes por bloques grandes y solo recurre a separadores más finos si no queda otra. Cada chunk recibe un ID derivado del registro original y metadatos ricos que luego permiten filtrar y presentar fuentes en la app.

7.7 Utilidades de JSON y manifiestos

`write_json(path, payload)`

Crea la carpeta si no existe y serializa el contenido con indentación y UTF-8.

`read_json(path)`

Lee un JSON desde disco y lo materializa como estructura Python.

`load_clean_records(path)`

Asegura que el archivo limpio contiene realmente una lista. Si no, lanza un error explícito.

`load_index_manifest(vectorstore_dir)`

Intenta leer el manifiesto de indexación. Si aún no existe, devuelve `None`. La app usa esta convención para decidir si puede arrancar.

7.8 Seguridad y directorios

`ensure_safe_output_dir(path, project_root)`

Protege frente a borrados peligrosos. Resuelve la ruta final, comprueba que está dentro del proyecto y que no coincide con la raíz del mismo. Solo entonces elimina el contenido previo y recrea el directorio. Esta pequeña función evita errores graves de automatización.

7.9 Descubrimiento de modelos de Ollama

`list_installed_ollama_models()`

Ejecuta `ollama list` como subprocesso y parsea la salida para devolver los nombres de modelos instalados localmente.

`pick_ollama_model(preferred_names, fallback)`

Intenta encontrar el mejor modelo disponible dentro de una lista preferida, primero por coincidencia exacta y luego por prefijo, usando un fallback si no encuentra ninguno.

`resolve_default_llm_model()` y `resolve_default_embedding_model()`

Estas funciones priorizan variables de entorno, luego metadatos ya conocidos del sistema y, finalmente, un mecanismo de autodetección basado en modelos instalados. Gracias a esto, la app arranca con menos fricción en máquinas distintas.

7.10 Formateo del contexto recuperado

`build_context_block(results)`

Convierte la lista de fragmentos recuperados en un bloque de texto estructurado para el prompt del LLM. Cada fuente incluye título, tema, relevancia, URL y contenido.

`format_sources_markdown(results)`

Genera la versión legible por humanos de esas mismas fuentes para mostrarla en Gradio. Incluye extractos, puntuación y metadatos visibles en la interfaz.

8. Explicación completa de `app.py`

Este archivo une todo el sistema. Carga configuración, conecta con el índice vectorial, define el prompt del asistente, construye el pipeline RAG y monta la interfaz de usuario.

8.1 Arranque y configuración global

- `load_dotenv()` permite que el comportamiento del sistema cambie mediante un archivo `.env`.
- `PROJECT_ROOT` y `VECTORSTORE_DIR` anclan rutas relativas al directorio del proyecto.
- `MANIFEST` intenta cargar el manifiesto del índice ya creado.

- `OLLAMA_BASE_URL`, `DEFAULT_LLM_MODEL` y `DEFAULT_EMBED_MODEL` se resuelven combinando entorno, manifiesto y autodetección.

8.2 SYSTEM_PROMPT

Es una pieza crítica del comportamiento del sistema. Define restricciones duras:

- responder solo con contexto recuperado
- declarar explícitamente cuando la respuesta no aparece
- no usar conocimiento externo
- responder en español por defecto
- citar fuentes usando la numeración del contexto

Estas reglas son el principal control de alucinaciones a nivel de prompt.

8.3 PROMPT

Se construye con `ChatPromptTemplate.from_messages()`. La plantilla combina tres piezas:

1. mensaje de sistema con el contexto recuperado
2. historial previo de conversación
3. pregunta actual del usuario

Eso permite preguntas de seguimiento sin perder el hilo conversacional.

8.4 CUSTOM_CSS

Esta cadena CSS personaliza la experiencia visual. No es solo decoración; también organiza la jerarquía visual y mejora legibilidad. Define colores, tipografía, paneles, botones y contenedores de la interfaz. Todo se inyecta al lanzar Gradio.

8.5 Estado global

- `INITIAL_SOURCES` es el texto visible antes de la primera búsqueda.
- `_VECTORSTORE` guarda una referencia singleton al índice Chroma para no recrearlo en cada interacción.

8.6 get_topics()

Construye la lista de temas disponibles para el desplegable de filtros. Siempre incluye la opción Todos y, si existe un manifiesto, añade los temas almacenados en él.

8.7 get_vectorstore()

Implementa carga perezosa del índice. Si ya existe en memoria, lo devuelve directamente. Si no existe y no hay manifiesto, lanza un error guiado. En caso contrario, crea el objeto `OllamaEmbeddings`, abre la colección Chroma persistente y la cachea en la variable global.

8.8 to_langchain_history(history)

Convierte la estructura de historial que maneja Gradio en la estructura de mensajes que entiende LangChain. Mapea diccionarios con rol user o assistant a objetos HumanMessage y AIMessage.

8.9 retrieve_chunks(question, k_value, score_threshold, topic_filter)

Resuelve la recuperación semántica. Toma una consulta, aplica opcionalmente un filtro por tema, llama a `similarity_search_with_relevance_scores()` y filtra resultados por umbral. La normalización del score a rango 0–1 permite que el umbral de la UI tenga un significado estable.

8.10 answer_question(...)

Es la función más importante de la app. Gestiona todo el ciclo de una pregunta.

1. Normaliza el historial y la pregunta.
2. Si la pregunta está vacía, no hace nada y devuelve el estado intacto.
3. Intenta recuperar fragmentos con `retrieve_chunks()`.
4. Si falla la recuperación, devuelve un mensaje de error a la interfaz.
5. Si no hay fragmentos, responde que no hay contexto suficiente.
6. Si sí hay fragmentos, crea una cadena LangChain compuesta por prompt, modelo y parser.
7. Construye el bloque de contexto con `build_context_block()`.
8. Invoca el modelo con historial y pregunta.
9. Si el modelo devuelve una respuesta que huele a “no tengo contexto”, el código fuerza una frase estándar para evitar que añada conocimiento externo.
10. Finalmente actualiza el historial y devuelve también el markdown de fuentes y el textbox vacío.

Punto especialmente importante: esta función contiene una segunda capa de defensa contra alucinaciones. No se limita a confiar en el prompt; inspecciona la respuesta del modelo y la normaliza si detecta patrones de insuficiencia de contexto.

8.11 clear_chat()

Reinicia por completo el estado visible del chat: historial, fuentes y caja de texto.

8.12 build_status_markdown()

Genera el panel de estado de la interfaz. Si no existe un índice, muestra instrucciones. Si sí existe, enseña datos operativos del sistema, como número de chunks, modelo de embeddings y ubicación del manifiesto.

8.13 build_demo()

Construye toda la UI con la API declarativa de Gradio:

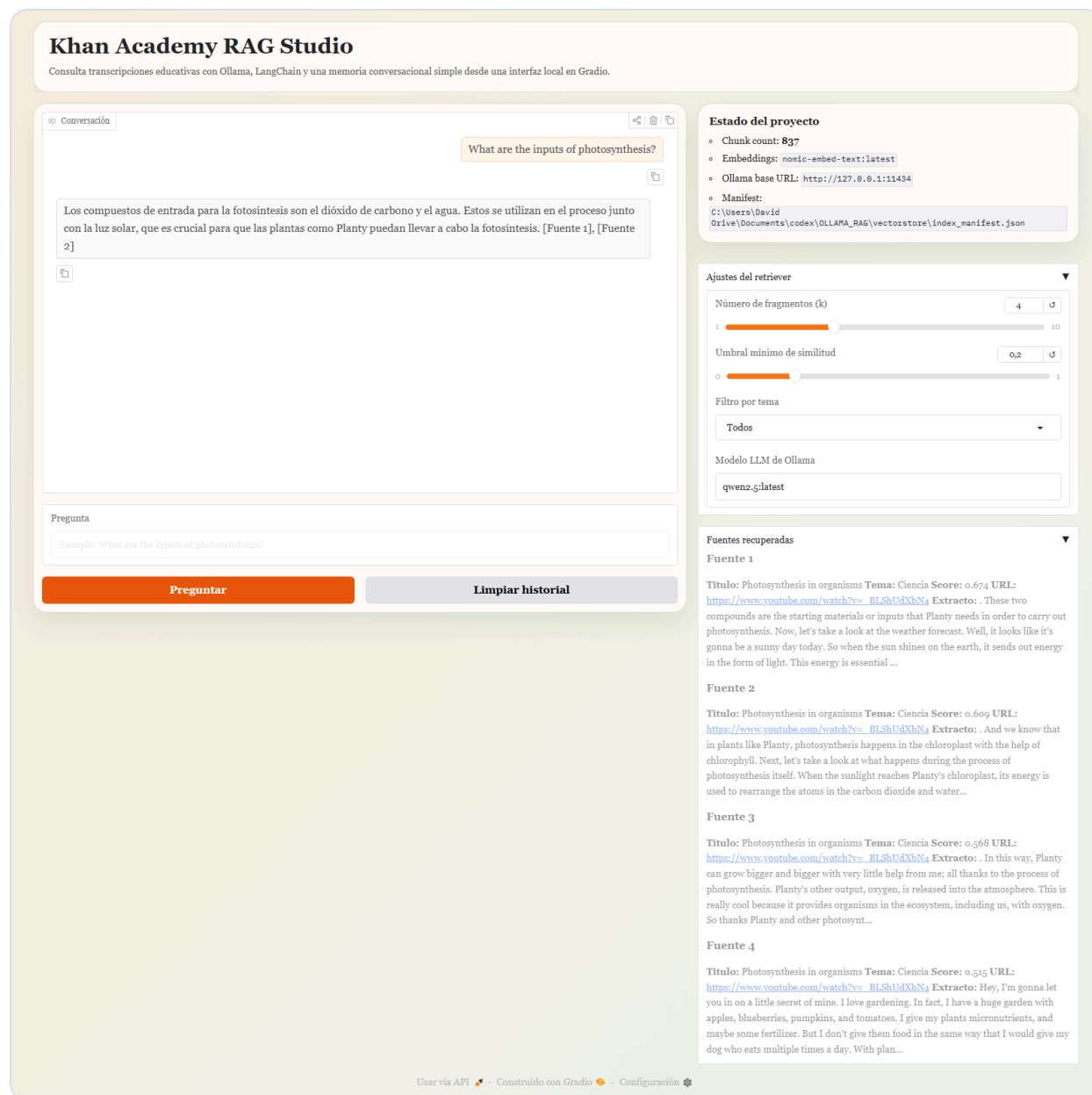
- usa `Blocks` como contenedor principal
- muestra una cabecera de tipo "hero"
- crea una columna de chat con `Chatbot`, `State`, caja de pregunta y botones
- crea una segunda columna con el estado, ajustes del retriever y fuentes recuperadas
- declara explícitamente las entradas y salidas de la función `answer_question()`
- conecta eventos de `submit`, `click` y `limpieza`

Esta función es importante porque deja la lógica de interfaz bien encapsulada: fuera de ella, el resto del código no necesita saber nada de widgets concretos.

8.14 Punto de entrada

El bloque `if __name__ == "__main__"` lanza la app con cola de eventos y CSS personalizado. Así el archivo sirve tanto como módulo importable como aplicación ejecutable.

Captura de la interfaz en funcionamiento



La imagen muestra cómo el código de `app.py` traduce el índice vectorial y el historial de conversación a una experiencia de usuario usable.

9. Decisiones de diseño, límites y mejoras posibles

9.1 Decisiones acertadas del proyecto

- Separar limpieza, indexación y aplicación en scripts distintos evita acoplamiento innecesario.
- Centralizar utilidades en `rag_utils.py` reduce duplicación y mejora mantenibilidad.
- Persistir Chroma en disco evita reindexar cada vez.
- Usar un manifiesto de índice permite a la app autoexplicarse y detectar configuración previa.

- Introducir un filtro por tema añade control al usuario con una heurística barata.
- Añadir una segunda barrera contra alucinaciones en `answer_question()` mejora robustez.

9.2 Límites actuales

- La clasificación temática es heurística, no semántica.
- El `score threshold` depende del comportamiento de la `vector store` y del `embedding` elegido.
- La memoria conversacional usa el historial visible, no una memoria resumida o jerárquica.
- No hay tests automáticos en el repositorio actual.
- La explicación de fuentes es clara, pero no incluye trazabilidad exacta de `chunk IDs` en la interfaz.

9.3 Mejoras razonables

- añadir tests unitarios para `clean_transcript()`, `infer_topic()` y `normalize_record()`
- permitir persistir conversaciones o exportarlas desde la app
- añadir re-ranking o compresión contextual antes del LLM
- mostrar metadatos extra, como número de `chunk` y `score`, de forma más visible en la UI
- incorporar una fase de evaluación automatizada de respuestas RAG

10. Anexo: código fuente completo

Para que la explicación sea realmente exhaustiva, el documento incluye a continuación el contenido completo de todos los archivos fuente y de configuración principales del proyecto. Esto permite cruzar la explicación anterior con el código real línea por línea.

.gitignore

```
.venv/  
__pycache__/  
.pytest_cache/  
vectorstore/  
.env
```

.env.example

```
OLLAMA_BASE_URL=http://127.0.0.1:11434  
OLLAMA_LLM_MODEL=qwen2.5:latest  
OLLAMA_EMBED_MODEL=nomic-embed-text:latest
```

requirements.txt

```
chromadb==1.5.8
datasets==4.8.4
gradio==6.12.0
langchain==1.2.15
langchain-chroma==1.1.0
langchain-community==0.4.1
langchain-ollama==1.1.0
langchain-text-splitters==1.1.2
python-dotenv==1.2.2
```


clean_json.py

```
from __future__ import annotations

import argparse
from collections import Counter
from pathlib import Path

from datasets import load_dataset

from rag_utils import (
    DEFAULT_CLEAN_PATH,
    DEFAULT_DATASET_NAME,
    DEFAULT_DATASET_SPLIT,
    DEFAULT_MAX_RECORDS,
    DEFAULT_MIN_WORDS,
    choose_balanced_subset,
    deduplicate_records,
    load_clean_records,
    normalize_record,
    read_json,
    write_json,
)

def parse_args() -> argparse.Namespace:
    parser = argparse.ArgumentParser(
        description="Clean Khan Academy transcripts for the Ollama + LangChain RAG project."
    )
    parser.add_argument("--dataset-name", default=DEFAULT_DATASET_NAME)
    parser.add_argument("--split", default=DEFAULT_DATASET_SPLIT)
    parser.add_argument(
        "--input-json",
        type=Path,
        default=None,
        help="Optional local JSON file instead of downloading from Hugging Face.",
    )
    parser.add_argument("--output", type=Path, default=DEFAULT_CLEAN_PATH)
    parser.add_argument("--max-records", type=int, default=DEFAULT_MAX_RECORDS)
    parser.add_argument("--min-words", type=int, default=DEFAULT_MIN_WORDS)
    parser.add_argument("--seed", type=int, default=42)
    return parser.parse_args()

def load_source_records(args: argparse.Namespace) -> list[dict]:
    if args.input_json:
        payload = read_json(args.input_json)
        if isinstance(payload, list):
            return payload
        if isinstance(payload, dict) and isinstance(payload.get("train"), list):
            return payload["train"]
        raise ValueError(f"Unsupported JSON structure in {args.input_json}")

    dataset = load_dataset(args.dataset_name, split=args.split)
    return list(dataset)

def main() -> None:
    args = parse_args()
    raw_records = load_source_records(args)
```

```

cleaned_records = []
for raw_record in raw_records:
    normalized = normalize_record(raw_record, min_words=args.min_words)
    if normalized is not None:
        cleaned_records.append(normalized)

deduped_records = deduplicate_records(cleaned_records)
selected_records = choose_balanced_subset(
    deduped_records,
    max_records=args.max_records,
    seed=args.seed,
)
write_json(args.output, selected_records)

topic_counts = Counter(record["topic"] for record in selected_records)

print(f"Raw records loaded: {len(raw_records)}")
print(f"Records after cleaning: {len(cleaned_records)}")
print(f"Records after deduplication: {len(deduped_records)}")
print(f"Records written to {args.output.resolve()}: {len(selected_records)}")
print("Topic distribution:")
for topic, count in sorted(topic_counts.items()):
    print(f"  - {topic}: {count}")

if __name__ == "__main__":
    main()

```

index_data.py

```
from __future__ import annotations

import argparse
import time
from pathlib import Path

from langchain_chroma import Chroma
from langchain_ollama import OllamaEmbeddings

from rag_utils import (
    DEFAULT_CHUNK_OVERLAP,
    DEFAULT_CHUNK_SIZE,
    DEFAULT_CLEAN_PATH,
    DEFAULT_COLLECTION_NAME,
    DEFAULT_MANIFEST_NAME,
    DEFAULT_OLLAMA_BASE_URL,
    DEFAULT_VECTORSTORE_DIR,
    build_documents,
    ensure_safe_output_dir,
    load_clean_records,
    resolve_default_embedding_model,
    write_json,
)

def parse_args() -> argparse.Namespace:
    parser = argparse.ArgumentParser(
        description="Build a local Chroma vector store for the Khan Academy RAG project."
    )
    parser.add_argument("--input", type=Path, default=DEFAULT_CLEAN_PATH)
    parser.add_argument("--persist-dir", type=Path, default=DEFAULT_VECTORSTORE_DIR)
    parser.add_argument("--collection-name", default=DEFAULT_COLLECTION_NAME)
    parser.add_argument("--chunk-size", type=int, default=DEFAULT_CHUNK_SIZE)
    parser.add_argument("--chunk-overlap", type=int, default=DEFAULT_CHUNK_OVERLAP)
    parser.add_argument("--embedding-model", default=None)
    parser.add_argument("--ollama-base-url", default=DEFAULT_OLLAMA_BASE_URL)
    return parser.parse_args()

def main() -> None:
    args = parse_args()

    if not args.input.exists():
        raise SystemExit(
            f"Clean dataset not found at {args.input.resolve()}. Run clean_json.py first."
        )

    records = load_clean_records(args.input)
    documents = build_documents(
        records,
        chunk_size=args.chunk_size,
        chunk_overlap=args.chunk_overlap,
    )
    embedding_model = args.embedding_model or resolve_default_embedding_model()

    ensure_safe_output_dir(args.persist_dir, Path.cwd())
    embeddings = OllamaEmbeddings(
        model=embedding_model,
```

```

        base_url=args.ollama_base_url,
    )
    vectorstore = Chroma(
        collection_name=args.collection_name,
        persist_directory=str(args.persist_dir),
        embedding_function=embeddings,
    )

    try:
        for start in range(0, len(documents), 64):
            batch = documents[start : start + 64]
            vectorstore.add_documents(batch, ids=[doc.id for doc in batch])
    except Exception as exc:
        raise SystemExit(
            "Could not create the vector index. Verify that Ollama is running and "
            "that the embedding model is available. Suggested command: "
            "`ollama pull nomic-embed-text:latest`"
        ) from exc

    manifest = {
        "created_at": time.strftime("%Y-%m-%dT%H:%M:%SZ", time.gmtime()),
        "collection_name": args.collection_name,
        "dataset_path": str(args.input.resolve()),
        "record_count": len(records),
        "chunk_count": len(documents),
        "chunk_size": args.chunk_size,
        "chunk_overlap": args.chunk_overlap,
        "embedding_model": embedding_model,
        "ollama_base_url": args.ollama_base_url,
        "topics": sorted({record["topic"] for record in records}),
    }
    write_json(args.persist_dir / DEFAULT_MANIFEST_NAME, manifest)

    print(f"Clean records indexed: {len(records)}")
    print(f"Chunks stored in Chroma: {len(documents)}")
    print(f"Embedding model: {embedding_model}")
    print(f"Vector store directory: {args.persist_dir.resolve()}")

if __name__ == "__main__":
    main()

```

rag_utils.py

```
from __future__ import annotations

import hashlib
import json
import os
import random
import re
import shutil
import subprocess
from collections import defaultdict
from pathlib import Path
from typing import Any, Sequence

from langchain_core.documents import Document
from langchain_text_splitters import RecursiveCharacterTextSplitter

DEFAULT_DATASET_NAME = "iblai/ibl-khanacademy-transcripts"
DEFAULT_DATASET_SPLIT = "train"
DEFAULT_MIN_WORDS = 50
DEFAULT_MAX_RECORDS = 120
DEFAULT_CHUNK_SIZE = 1200
DEFAULT_CHUNK_OVERLAP = 200
DEFAULT_COLLECTION_NAME = "khanacademy_rag"
DEFAULT_CLEAN_PATH = Path("train_clean.json")
DEFAULT_VECTORSTORE_DIR = Path("vectorstore")
DEFAULT_MANIFEST_NAME = "index_manifest.json"
DEFAULT_OLLAMA_BASE_URL = "http://127.0.0.1:11434"
TOPIC_ALL_LABEL = "Todos"

TIMESTAMP_PATTERN = re.compile(
    r"^\d{2}:\d{2}:\d{2}\.\d{3}\s+-->\s+\d{2}:\d{2}:\d{2}\.\d{3}.*$"
)
CUE_INDEX_PATTERN = re.compile(r"^\d+$")
TAG_PATTERN = re.compile(r"^[^\]]+\]")
HTML_PATTERN = re.compile(r"<[>]+>")
WHITESPACE_PATTERN = re.compile(r"\s+")
SENTENCE_SPLIT_PATTERN = re.compile(r"(?<=[!?])\s+")

TOPIC_KEYWORDS = {
    "Matematicas": [
        "algebra",
        "geometry",
        "trigonometry",
        "calculus",
        "equation",
        "inequality",
        "fraction",
        "probability",
        "statistics",
        "ratio",
        "exponent",
        "linear",
        "quadratic",
        "function",
        "derivative",
        "integral",
        "triangle",
        "circle",
    ]
}
```

```
    "polynomial",
    "graph",
    "slope",
    "decimal",
    "percent",
  ],
  "Ciencia": [
    "biology",
    "chemistry",
    "physics",
    "energy",
    "fuel",
    "fossil",
    "cell",
    "atom",
    "molecule",
    "photosynthesis",
    "electricity",
    "force",
    "motion",
    "wave",
    "climate",
    "planet",
    "earth",
    "ecosystem",
    "reaction",
    "matter",
  ],
  "Economia y finanzas": [
    "economics",
    "finance",
    "inflation",
    "gdp",
    "market",
    "supply",
    "demand",
    "trade",
    "money",
    "bank",
    "interest",
    "revenue",
    "cost",
    "profit",
    "tax",
    "budget",
    "investment",
    "capital",
    "consumer",
    "producer",
  ],
  "Computacion": [
    "computer",
    "programming",
    "algorithm",
    "python",
    "javascript",
    "html",
    "css",
    "sql",
    "binary",
    "internet",
```

```
    "database",
    "debug",
    "coding",
    "code",
  ],
  "Lengua y vocabulario": [
    "grammar",
    "vocabulary",
    "sentence",
    "verb",
    "noun",
    "adjective",
    "pronoun",
    "punctuation",
    "reading",
    "writing",
    "essay",
    "poem",
    "synonym",
    "antonym",
    "prefix",
    "suffix",
    "wordsmith",
    "connotation",
  ],
  "Historia y civismo": [
    "history",
    "government",
    "constitution",
    "war",
    "revolution",
    "empire",
    "president",
    "democracy",
    "civil rights",
    "colonial",
    "ancient",
    "medieval",
    "state",
    "federal",
    "citizen",
    "geography",
  ],
  "Preparacion de examenes": [
    "sat",
    "lsat",
    "gmat",
    "test prep",
    "exam",
    "passage",
    "reading comprehension",
    "practice question",
  ],
  "Arte y humanidades": [
    "art",
    "music",
    "painting",
    "philosophy",
    "theater",
    "humanities",
    "story",
  ],
```

```

        "narrative",
    ],
}

def normalize_whitespace(text: str) -> str:
    return WHITESPACE_PATTERN.sub(" ", (text or "").strip())

def stable_text_id(*parts: str) -> str:
    digest = hashlib.sha1("||".join(parts).encode("utf-8")).hexdigest()
    return digest[:16]

def dedupe_adjacent_sentences(text: str) -> str:
    sentences = []
    previous = ""
    for sentence in SENTENCE_SPLIT_PATTERN.split(text):
        cleaned = normalize_whitespace(sentence)
        if not cleaned:
            continue
        lowered = cleaned.lower()
        if lowered == previous:
            continue
        sentences.append(cleaned)
        previous = lowered
    return " ".join(sentences)

def clean_transcript(raw_text: str) -> str:
    lines = []
    previous_line = ""

    for raw_line in (raw_text or "").replace("\r", "\n").split("\n"):
        line = raw_line.strip()
        if not line:
            continue
        if line.upper() == "WEBVTT":
            continue
        if line.lower().startswith("kind:"):
            continue
        if line.lower().startswith("language:"):
            continue
        if line.startswith("NOTE"):
            continue
        if TIMESTAMP_PATTERN.fullmatch(line):
            continue
        if CUE_INDEX_PATTERN.fullmatch(line):
            continue
        normalized = normalize_whitespace(line)
        if normalized.lower() == previous_line:
            continue
        lines.append(normalized)
        previous_line = normalized.lower()

    text = " ".join(lines)
    text = HTML_PATTERN.sub(" ", text)
    text = TAG_PATTERN.sub(" ", text)
    text = text.replace("&", "&")
    text = text.replace(""", "'")
    text = text.replace("&#39;", "'")

```



```

text = re.sub(r"^\s)-\s+", " ", text)
text = normalize_whitespace(text)
text = dedupe_adjacent_sentences(text)
return normalize_whitespace(text)

def infer_topic(title: str, content: str) -> str:
    title_lower = (title or "").lower()
    preview = (content or "")[:1800].lower()
    blob = f"{title_lower} {preview}"

    scores = {
        topic: sum(1 for keyword in keywords if keyword in blob)
        for topic, keywords in TOPIC_KEYWORDS.items()
    }
    best_topic = max(scores, key=scores.get)
    if scores[best_topic] > 0:
        return best_topic

    short_title = len(title.split()) <= 3
    language_markers = ["word", "noun", "verb", "adjective", "synonym", "meaning"]
    if short_title and any(marker in blob for marker in language_markers):
        return "Lengua y vocabulario"

    return "General"

def normalize_record(raw_record: dict[str, Any], min_words: int = DEFAULT_MIN_WORDS) -> dict[str, Any] | None:
    title = normalize_whitespace(str(raw_record.get("title", "")))
    content = clean_transcript(str(raw_record.get("content", "")))
    url = normalize_whitespace(
        str(raw_record.get("video_url") or raw_record.get("url") or "")
    )
    subtitle_url = normalize_whitespace(str(raw_record.get("url", "")))

    if not title or not content:
        return None

    word_count = len(content.split())
    if word_count < min_words:
        return None

    record_id = stable_text_id(title, url or content[:200])
    return {
        "id": record_id,
        "title": title,
        "content": content,
        "url": url,
        "subtitle_url": subtitle_url,
        "language": normalize_whitespace(str(raw_record.get("language", "en"))) or "en",
        "topic": infer_topic(title, content),
        "word_count": word_count,
    }

def deduplicate_records(records: list[dict[str, Any]]) -> list[dict[str, Any]]:
    deduped = []
    seen = set()

    for record in records:

```

```

        fingerprint = record["url"] or stable_text_id(
            record["title"], record["content"][:400]
        )
        if fingerprint in seen:
            continue
        seen.add(fingerprint)
        deduped.append(record)

    return deduped

def choose_balanced_subset(
    records: list[dict[str, Any]],
    max_records: int | None = DEFAULT_MAX_RECORDS,
    seed: int = 42,
) -> list[dict[str, Any]]:
    if not max_records or max_records <= 0 or len(records) <= max_records:
        return records

    grouped: dict[str, list[dict[str, Any]]] = defaultdict(list)
    for record in records:
        grouped[record["topic"]].append(record)

    rng = random.Random(seed)
    for topic_records in grouped.values():
        rng.shuffle(topic_records)

    ordered_topics = sorted(grouped)
    selected = []

    while len(selected) < max_records and any(grouped.values()):
        for topic in ordered_topics:
            if not grouped[topic]:
                continue
            selected.append(grouped[topic].pop())
            if len(selected) >= max_records:
                break

    return selected

def build_documents(
    records: list[dict[str, Any]],
    chunk_size: int = DEFAULT_CHUNK_SIZE,
    chunk_overlap: int = DEFAULT_CHUNK_OVERLAP,
) -> list[Document]:
    splitter = RecursiveCharacterTextSplitter(
        chunk_size=chunk_size,
        chunk_overlap=chunk_overlap,
        separators=["\n\n", "\n", ". ", " ", ""],
    )

    documents = []
    for record in records:
        chunks = splitter.split_text(record["content"])
        for index, chunk in enumerate(chunks, start=1):
            documents.append(
                Document(
                    id=f"{record['id']}-{index:03d}",
                    page_content=chunk,
                    metadata={

```

```

        "record_id": record["id"],
        "title": record["title"],
        "url": record["url"],
        "source": record["url"],
        "subtitle_url": record["subtitle_url"],
        "topic": record["topic"],
        "language": record["language"],
        "word_count": record["word_count"],
        "chunk_index": index,
        "chunk_total": len(chunks),
    },
)
)
return documents

def write_json(path: Path, payload: Any) -> None:
    path.parent.mkdir(parents=True, exist_ok=True)
    path.write_text(
        json.dumps(payload, indent=2, ensure_ascii=False),
        encoding="utf-8",
    )

def read_json(path: Path) -> Any:
    return json.loads(path.read_text(encoding="utf-8"))

def load_clean_records(path: Path) -> list[dict[str, Any]]:
    data = read_json(path)
    if not isinstance(data, list):
        raise ValueError(f"Expected a JSON list in {path}")
    return data

def load_index_manifest(vectorstore_dir: Path) -> dict[str, Any] | None:
    manifest_path = vectorstore_dir / DEFAULT_MANIFEST_NAME
    if not manifest_path.exists():
        return None
    return read_json(manifest_path)

def ensure_safe_output_dir(path: Path, project_root: Path) -> None:
    resolved_path = path.resolve()
    resolved_root = project_root.resolve()
    if resolved_path == resolved_root or not resolved_path.is_relative_to(resolved_root):
        raise ValueError(f"Unsafe output directory: {resolved_path}")
    if resolved_path.exists():
        shutil.rmtree(resolved_path)
    resolved_path.mkdir(parents=True, exist_ok=True)

def list_installed_ollama_models() -> list[str]:
    try:
        result = subprocess.run(
            ["ollama", "list"],
            check=True,
            capture_output=True,
            text=True,
            encoding="utf-8",
        )

```

```

except (FileNotFoundError, subprocess.CalledProcessError):
    return []

models = []
for line in result.stdout.splitlines()[1:]:
    parts = line.split()
    if parts:
        models.append(parts[0])
return models

def pick_ollama_model(preferred_names: Sequence[str], fallback: str) -> str:
    installed = list_installed_ollama_models()
    installed_lookup = {name.lower(): name for name in installed}

    for candidate in preferred_names:
        if candidate.lower() in installed_lookup:
            return installed_lookup[candidate.lower()]

    for candidate in preferred_names:
        root_name = candidate.replace(":latest", "").lower()
        for installed_name in installed:
            if installed_name.lower().startswith(root_name):
                return installed_name

    return fallback

def resolve_default_llm_model() -> str:
    env_model = os.getenv("OLLAMA_LLM_MODEL")
    if env_model:
        return env_model
    return pick_ollama_model(
        ["llama3.2:latest", "mistral:latest", "qwen2.5:latest"],
        "qwen2.5:latest",
    )

def resolve_default_embedding_model(manifest_model: str | None = None) -> str:
    env_model = os.getenv("OLLAMA_EMBED_MODEL")
    if env_model:
        return env_model
    if manifest_model:
        return manifest_model
    return pick_ollama_model(
        ["nomic-embed-text:latest", "nomic-embed-text"],
        "nomic-embed-text:latest",
    )

def build_context_block(results: list[tuple[Document, float]]) -> str:
    blocks = []
    for index, (doc, score) in enumerate(results, start=1):
        clipped_score = max(0.0, min(float(score), 1.0))
        blocks.append(
            "\n".join(
                [
                    f"[Fuente {index}]",
                    f"Titulo: {doc.metadata.get('title', 'Sin titulo')}",
                    f"Tema: {doc.metadata.get('topic', 'General')}",
                    f"Relevancia: {clipped_score:.3f}",
                ]
            )
        )

```

```

        f"URL: {doc.metadata.get('url', '')}",
        "Contenido:",
        doc.page_content,
    ]
)
)
return "\n\n".join(blocks)

```

```

def format_sources_markdown(results: list[tuple[Document, float]]) -> str:
    if not results:
        return "No se recuperaron fragmentos con los parametros actuales."

```

```

    entries = []
    for index, (doc, score) in enumerate(results, start=1):
        snippet = normalize_whitespace(doc.page_content)[:320]
        clipped_score = max(0.0, min(float(score), 1.0))
        entries.append(
            "\n".join(
                [
                    f"### Fuente {index}",
                    f"**Titulo:** {doc.metadata.get('title', 'Sin titulo')}",
                    f"**Tema:** {doc.metadata.get('topic', 'General')}",
                    f"**Score:** {clipped_score:.3f}",
                    f"**URL:** {doc.metadata.get('url', '')}",
                    f"**Extracto:** {snippet}...",
                ]
            )
        )
    return "\n\n".join(entries)

```

app.py

```
from __future__ import annotations

import os
from pathlib import Path

import gradio as gr
from dotenv import load_dotenv
from langchain_chroma import Chroma
from langchain_core.messages import AIMessage, HumanMessage
from langchain_core.output_parsers import StrOutputParser
from langchain_core.prompts import ChatPromptTemplate, MessagesPlaceholder
from langchain_ollama import ChatOllama, OllamaEmbeddings

from rag_utils import (
    DEFAULT_COLLECTION_NAME,
    DEFAULT_MANIFEST_NAME,
    DEFAULT_OLLAMA_BASE_URL,
    DEFAULT_VECTORSTORE_DIR,
    TOPIC_ALL_LABEL,
    build_context_block,
    format_sources_markdown,
    load_index_manifest,
    resolve_default_embedding_model,
    resolve_default_llm_model,
)

load_dotenv()

PROJECT_ROOT = Path(__file__).resolve().parent
VECTORSTORE_DIR = PROJECT_ROOT / DEFAULT_VECTORSTORE_DIR
MANIFEST = load_index_manifest(VECTORSTORE_DIR)
OLLAMA_BASE_URL = os.getenv("OLLAMA_BASE_URL") or (
    MANIFEST or {}
).get("ollama_base_url", DEFAULT_OLLAMA_BASE_URL)
DEFAULT_LLM_MODEL = resolve_default_llm_model()
DEFAULT_EMBED_MODEL = resolve_default_embedding_model(
    (MANIFEST or {}).get("embedding_model")
)

SYSTEM_PROMPT = """
Eres un asistente educativo especializado en transcripciones de Khan Academy.
Reglas obligatorias:
- Responde solo con el contexto recuperado.
- Si el contexto no basta o la respuesta no aparece de forma explícita, responde exactamente: "No encuentro esa información en los fragmentos recuperados."
- Nunca añadas explicaciones generales, ejemplos externos ni conocimiento propio después de indicar que falta contexto.
- No inventes datos ni uses conocimiento externo.
- Responde en español salvo que el usuario pida otro idioma.
- Cuando apoyes una idea en el contexto, cita [Fuente 1], [Fuente 2], etc.

Contexto recuperado:
{context}
""".strip()

PROMPT = ChatPromptTemplate.from_messages(
    [
        ("system", SYSTEM_PROMPT),
```

```

        MessagesPlaceholder("history"),
        ("human", "{question}"),
    ]
)

CUSTOM_CSS = """
:root {
  --page-bg: linear-gradient(135deg, #f7efe0 0%, #edf4ea 100%);
  --panel-bg: rgba(255, 252, 246, 0.94);
  --card-bg: rgba(255, 255, 255, 0.86);
  --ink: #1f2d1f;
  --muted: #55645b;
  --accent: #2d6a4f;
  --accent-2: #c56b3a;
  --border: rgba(45, 106, 79, 0.18);
}

.gradio-container {
  background: var(--page-bg);
  color: var(--ink);
  font-family: Georgia, "Times New Roman", serif;
}

#hero-card,
#side-note,
.panel-shell {
  background: var(--panel-bg);
  border: 1px solid var(--border);
  border-radius: 20px;
  box-shadow: 0 18px 36px rgba(31, 45, 31, 0.08);
}

#hero-card {
  padding: 18px 22px;
  margin-bottom: 18px;
}

#hero-card h1 {
  margin: 0 0 8px 0;
  font-size: 2rem;
  line-height: 1.1;
}

#hero-card p {
  margin: 0;
  color: var(--muted);
}

#side-note {
  padding: 16px;
  margin-bottom: 14px;
}

.panel-shell {
  padding: 12px;
}

.gr-button-primary {
  background: linear-gradient(135deg, var(--accent), #4d8f6d) !important;
  border: none !important;
}

```

```

.gr-button-secondary {
    border: 1px solid var(--border) !important;
}
"""

INITIAL_SOURCES = "Las fuentes recuperadas aparecerán aquí después de la primera pregunta."

_VECTORSTORE: Chroma | None = None

def get_topics() -> list[str]:
    topics = [TOPIC_ALL_LABEL]
    if MANIFEST and MANIFEST.get("topics"):
        topics.extend(MANIFEST["topics"])
    return topics

def get_vectorstore() -> Chroma:
    global _VECTORSTORE

    if _VECTORSTORE is not None:
        return _VECTORSTORE

    if MANIFEST is None:
        raise RuntimeError(
            "No se encontró un índice vectorial local. Ejecuta `python index_data.py` antes de lanzar la app."
        )

    embeddings = OllamaEmbeddings(
        model=DEFAULT_EMBED_MODEL,
        base_url=OLLAMA_BASE_URL,
    )
    _VECTORSTORE = Chroma(
        collection_name=MANIFEST.get("collection_name", DEFAULT_COLLECTION_NAME),
        persist_directory=str(VECTORSTORE_DIR),
        embedding_function=embeddings,
    )
    return _VECTORSTORE

def to_langchain_history(history: list[dict]) -> list[HumanMessage | AIMessage]:
    messages: list[HumanMessage | AIMessage] = []
    for item in history or []:
        role = item.get("role")
        content = str(item.get("content", "")).strip()
        if not content:
            continue
        if role == "user":
            messages.append(HumanMessage(content=content))
        elif role == "assistant":
            messages.append(AIMessage(content=content))
    return messages

def retrieve_chunks(
    question: str,
    k_value: int,
    score_threshold: float,
    topic_filter: str,

```



```

) -> list[tuple]:
    vectorstore = get_vectorstore()
    metadata_filter = None
    if topic_filter and topic_filter != TOPIC_ALL_LABEL:
        metadata_filter = {"topic": topic_filter}

    raw_results = vectorstore.similarity_search_with_relevance_scores(
        question,
        k=int(k_value),
        filter=metadata_filter,
    )

    filtered_results = []
    for doc, score in raw_results:
        normalized_score = max(0.0, min(float(score), 1.0))
        if normalized_score >= float(score_threshold):
            filtered_results.append((doc, normalized_score))

    return filtered_results

def answer_question(
    question: str,
    history: list[dict],
    k_value: int,
    score_threshold: float,
    topic_filter: str,
    llm_model: str,
):
    history = list(history or [])
    clean_question = (question or "").strip()

    if not clean_question:
        return history, history, INITIAL_SOURCES, ""

    try:
        retrieved = retrieve_chunks(
            clean_question,
            k_value=k_value,
            score_threshold=score_threshold,
            topic_filter=topic_filter,
        )
    except Exception as exc:
        answer = (
            "No pude consultar el índice vectorial. Comprueba que el índice local "
            "existe y que Ollama está activo.\n\n"
            f"Detalle: {exc}"
        )
        updated_history = history + [
            {"role": "user", "content": clean_question},
            {"role": "assistant", "content": answer},
        ]
        return updated_history, updated_history, INITIAL_SOURCES, ""

    if not retrieved:
        answer = (
            "No encuentro contexto suficiente en las transcripciones seleccionadas "
            "para responder con seguridad. Prueba a reformular la pregunta, bajar "
            "el umbral o cambiar el filtro de tema."
        )
        sources_markdown = "No se recuperaron fragmentos con el umbral actual."

```

```

else:
    model_name = (llm_model or DEFAULT_LLM_MODEL).strip() or DEFAULT_LLM_MODEL
    chain = PROMPT | ChatOllama(
        model=model_name,
        base_url=OLLAMA_BASE_URL,
        temperature=0.1,
    ) | StrOutputParser()
    context_block = build_context_block(retrieved)

    try:
        answer = chain.invoke(
            {
                "context": context_block,
                "history": to_langchain_history(history),
                "question": clean_question,
            }
        )
    except Exception as exc:
        answer = (
            f"No pude generar la respuesta con el modelo `{model_name}`. "
            "Verifica que el modelo exista en Ollama.\n\n"
            f"Detalle: {exc}"
        )

    lower_answer = answer.lower()
    insufficient_markers = [
        "no encuentro",
        "no hay suficiente informacion",
        "no dispongo de informacion",
        "el contexto no contiene",
        "the context does not contain",
    ]
    if any(marker in lower_answer for marker in insufficient_markers):
        answer = "No encuentro esa información en los fragmentos recuperados."

    sources_markdown = format_sources_markdown(retrieved)

    updated_history = history + [
        {"role": "user", "content": clean_question},
        {"role": "assistant", "content": answer},
    ]
    return updated_history, updated_history, sources_markdown, ""

def clear_chat():
    return [], [], INITIAL_SOURCES, ""

def build_status_markdown() -> str:
    if MANIFEST is None:
        return (
            "### Estado del proyecto\n"
            "- Índice vectorial: pendiente\n"
            "- Ejecuta `python index_data.py` para crear el índice.\n"
            f"- Modelo LLM por defecto: `{DEFAULT_LLM_MODEL}`\n"
            f"- Modelo de embeddings esperado: `{DEFAULT_EMBED_MODEL}`"
        )

    return (
        "### Estado del proyecto\n"
        f"- Chunk count: **{MANIFEST.get('chunk_count', 0)}**\n"
    )

```

```

f"- Embeddings: `{MANIFEST.get('embedding_model', DEFAULT_EMBED_MODEL)}`\n"
f"- Ollama base URL: `{OLLAMA_BASE_URL}`\n"
f"- Manifest: `{VECTORSTORE_DIR / DEFAULT_MANIFEST_NAME}`"
)

def build_demo() -> gr.Blocks:
    with gr.Blocks(
        title="Khan Academy RAG con Ollama",
        fill_width=True,
    ) as demo:
        gr.Markdown(
            """
<div id="hero-card">
    <h1>Khan Academy RAG Studio</h1>
    <p>Consulta transcripciones educativas con Ollama, LangChain y una memoria conversacional simple desde una interfaz local en Gradio.</p>
</div>
""").strip()
        )

        with gr.Row():
            with gr.Column(scale=5, elem_classes="panel-shell"):
                chatbot = gr.Chatbot(
                    label="Conversación",
                    height=560,
                    layout="bubble",
                    placeholder="Haz una pregunta sobre las transcripciones del dataset limpio.",
                )
                history_state = gr.State([])
                question_box = gr.Textbox(
                    label="Pregunta",
                    placeholder="Ejemplo: What are the inputs of photosynthesis?",
                )
            with gr.Row():
                send_button = gr.Button("Preguntar", variant="primary")
                clear_button = gr.Button("Limpiar historial", variant="secondary")

        with gr.Column(scale=3):
            gr.Markdown(build_status_markdown(), elem_id="side-note")
            with gr.Accordion("Ajustes del retriever", open=True):
                k_slider = gr.Slider(
                    minimum=1,
                    maximum=10,
                    value=4,
                    step=1,
                    label="Número de fragmentos (k)",
                )
                threshold_slider = gr.Slider(
                    minimum=0.0,
                    maximum=1.0,
                    value=0.2,
                    step=0.05,
                    label="Umbral mínimo de similitud",
                )
                topic_dropdown = gr.Dropdown(
                    choices=get_topics(),
                    value=TOPIC_ALL_LABEL,
                    label="Filtro por tema",
                )
                llm_model_box = gr.Textbox(

```

```

        value=DEFAULT_LLM_MODEL,
        label="Modelo LLM de Ollama",
    )

    with gr.Accordion("Fuentes recuperadas", open=True):
        sources_markdown = gr.Markdown(INITIAL_SOURCES)

    inputs = [
        question_box,
        history_state,
        k_slider,
        threshold_slider,
        topic_dropdown,
        llm_model_box,
    ]
    outputs = [chatbot, history_state, sources_markdown, question_box]

    question_box.submit(answer_question, inputs=inputs, outputs=outputs)
    send_button.click(answer_question, inputs=inputs, outputs=outputs)
    clear_button.click(
        clear_chat,
        inputs=None,
        outputs=[chatbot, history_state, sources_markdown, question_box],
    )

    return demo

if __name__ == "__main__":
    build_demo().queue().launch(css=CUSTOM_CSS)

```

README.md

Sistema RAG con Ollama, LangChain y Gradio

Proyecto completo para la tarea de RAG sobre transcripciones de Khan Academy usando:

- limpieza reproducible del dataset
- embeddings locales con Ollama
- base de datos vectorial Chroma persistida en disco
- pipeline RAG con memoria conversacional básica
- interfaz local con Gradio

Estructura

- `clean\_json.py`: descarga y limpia el dataset, y genera `train\_clean.json`
- `index\_data.py`: divide el contenido en chunks, crea embeddings y construye el índice vectorial local
- `app.py`: interfaz Gradio + pipeline RAG
- `rag\_utils.py`: utilidades compartidas
- `train\_clean.json`: dataset limpio en formato JSON, con un subconjunto balanceado por tema
- `vectorstore/`: índice persistente generado por `index\_data.py`
- `screenshots/`: capturas reales del proyecto en funcionamiento

Requisitos

- Python 3.11 recomendado
- Ollama instalado y en ejecución local
- Al menos un modelo generativo local en Ollama
- El modelo de embeddings `nomic-embed-text`

Modelos recomendados:

- `ollama pull nomic-embed-text:latest`
- `ollama pull llama3.2:latest`

En esta máquina también funciona con `qwen2.5:latest` como LLM local.

Instalación

PowerShell

```
```powershell
py -3.11 -m venv .venv
.\.venv\Scripts\Activate.ps1
pip install -r requirements.txt
```
```

Flujo de ejecución

1. Limpiar y preparar el dataset

```
```powershell
python clean_json.py --max-records 120
```
```

Notas:

- El script descarga `iblai/ibl-khanacademy-transcripts` desde Hugging Face si no indicas `--input-json`.
- Las transcripciones vacías o demasiado cortas se descartan.

- Se eliminan cabeceras VTT, timestamps, ruido y espacios sobrantes.
- El campo `url` final apunta al vídeo (`video\_url`) y se conserva la URL original de subtítulos en `subtitle\_url`.
- El campo `topic` se infiere para poder usar el filtro temático en la interfaz.

2. Crear el índice vectorial

```
```powershell
python index_data.py
```
```

Si quieres forzar un modelo concreto para embeddings:

```
```powershell
python index_data.py --embedding-model nomic-embed-text:latest
```
```

3. Lanzar la interfaz Gradio

```
```powershell
python app.py
```
```

La aplicación incluye:

- chat con historial conversacional
- control deslizante para `k`
- control deslizante para el umbral de similitud
- desplegable para filtrar por tema
- panel con los fragmentos recuperados
- botón para limpiar el historial

Variables de entorno opcionales

Puedes definir estas variables si quieres personalizar los modelos o la URL de Ollama:

- `OLLAMA\_BASE\_URL`
- `OLLAMA\_LLM\_MODEL`
- `OLLAMA\_EMBED\_MODEL`

Ejemplo:

```
```powershell
$env:OLLAMA_LLM_MODEL="qwen2.5:latest"
$env:OLLAMA_EMBED_MODEL="nomic-embed-text:latest"
python app.py
```
```

Capturas del proyecto funcionando

1. Vista inicial de la interfaz

![Vista inicial de la aplicación](screenshots/01-inicio.png)

2. Respuesta con contexto recuperado

Pregunta utilizada: `What are the inputs of photosynthesis?`

![Respuesta basada en contexto recuperado](screenshots/02-respuesta-con-contexto.png)

3. Caso sin información suficiente en los fragmentos recuperados

Pregunta utilizada: `What is the difference between fossil fuels and renewable energy?`

![Respuesta cuando el sistema no encuentra suficiente información](screenshots/03-sin-informacion-suficiente.png)

Comentarios técnicos

- Se usa `Chroma` persistido en la carpeta `vectorstore/` para no reindexar en cada arranque.
- La memoria conversacional se mantiene a través del historial del chat visible en la interfaz.
- El prompt obliga al modelo a responder solo con el contexto recuperado y a reconocer cuando no hay información suficiente.
- El dataset limpio se genera como un subconjunto balanceado por tema para mantener tiempos razonables durante el desarrollo.

Recursos

- Dataset: <https://huggingface.co/datasets/iblai/ib1-khanacademy-transcripts>
- LangChain RAG tutorial: <https://python.langchain.com/docs/tutorials/rag/>
- Gradio docs: <https://www.gradio.app/docs/>
- Chroma docs: <https://docs.trychroma.com/>